

Competitive Programming

Section 38: Static Queries and Binary Lifting

Competitive Programming Course

2026-07-10

Table of contents

1	Static query preprocessing	3
2	Sparse table worked query	4
3	Binary lifting	5
4	Recognition	6
5	Local exercises	7
6	How to use the checker	8
7	Static means a different tradeoff	9
7.1	Second sparse-table trace	9
8	Binary lifting as a path sparse table	10
8.1	Trace: kth ancestor	10
9	Notes, mistakes, and formative checks	11
10	Submission workflow	12

1 Static query preprocessing

When the data never changes, preprocessing can replace a dynamic data structure. Sparse tables answer idempotent range queries in $O(1)$ after $O(n \log n)$ preprocessing.

An operation is idempotent when:

```
op(x, x) = x
```

Minimum and gcd are idempotent. Sum is not.

Store:

```
st[k][i] = answer for range [i, i + 2k - 1]
```

Build by joining two half-length blocks:

```
st[k][i] = op(st[k-1][i], st[k-1][i + 2(k-1)])
```

A query $[l, r]$ has length len . Let $k = \text{floor}(\log_2(len))$. Two overlapping blocks of length 2^k cover the range:

```
[l, l+2k-1] and [r-2k+1, r]
```

Overlap is fine because the operation is idempotent.

2 Sparse table worked query

Array:

```
index: 1  2  3  4  5  6
value: 5  1  4  7  2  9
```

Query minimum on $[2,5]$. The length is 4, so $k = 2$ and one block of length 4 exactly covers the range:

```
st[2][2] = min([2,5])
```

Query minimum on $[2,6]$. The length is 5, so $k = 2$, block length 4. Use:

```
[2,5] and [3,6]
```

They overlap, but $\min(x,x)=x$, so overlap does not change the answer. For sums this would double-count the overlap, which is why sparse tables do not answer ordinary range sums this way.

3 Binary lifting

Binary lifting stores jumps of length powers of two:

```
up[k][v] = 2k-th ancestor of v
```

Recurrence:

```
up[k][v] = up[k-1][ up[k-1][v] ]
```

To move up d steps, decompose d in binary and take the corresponding jumps. This is the same power-of-two idea as sparse tables, applied to paths instead of intervals.

Example:

```
d = 13 = 8 + 4 + 1
```

To move a node up by 13, take jumps:

```
up[3], then up[2], then up[0]
```

The order does not matter for pure ancestors as long as every selected bit is used exactly once.

Minimal C++ jump:

```
int jump(int v, int d) {
    for (int k = 0; d; k++, d >>= 1) {
        if (d & 1) v = up[k][v];
        if (v == -1) break;
    }
    return v;
}
```

Minimal Python jump:

```
def jump(v, d):
    k = 0
    while d and v != -1:
        if d & 1:
            v = up[k][v]
        d >>= 1
        k += 1
    return v
```

4 Recognition

Static range min/gcd/max? Sparse table.
Static range sum? Prefix sums instead.
Repeated ancestor jumps? Binary lifting.
Need updates? Segment tree or Fenwick, not sparse table.

5 Local exercises

- A. Static Range Minimum: Given an immutable array, answer minimum queries on subarrays.
- B. Static Range GCD: Given an immutable positive array, answer gcd queries on subarrays.
- C. Kth Ancestor: In a rooted tree with root 1, answer queries asking for the k -th ancestor of a node, or -1 if it does not exist.

6 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Exercises

Use A and B to check the idempotence requirement of sparse-table queries, and C to define `up[j][v]` before lifting bits of `k`. Test a length-one range and an ancestor above the root.

7 Static means a different tradeoff

The signal is not merely “many queries”; it is “many queries and no updates.” For a static range minimum or gcd, spend $O(n \log n)$ memory and preprocessing to make every query $O(1)$. A sparse table’s invariant is:

```
st[k][i] is the aggregate of exactly [i, i+2k-1].
```

The build recurrence joins two adjacent valid half-blocks, so induction proves the invariant. A query uses two blocks of length $2^{\lfloor \log_2(\text{len}) \rfloor}$; their union covers the range. Their overlap is legal only for idempotent operations: $\min(x, x) = x$ and $\gcd(x, x) = x$, but $\text{sum}(x, x)$ double-counts.

7.1 Second sparse-table trace

For $[12, 18, 6, 10, 15]$, query gcd $[2, 5]$ has length four, so use one length-4 block and answer 1? No: $\gcd(18, 6, 10, 15) = 1$, which is exactly $\text{st}[2][2]$ in 1-based notation. For a length-five query, use two length-four overlapping blocks; applying gcd to both remains correct because duplicate values do not change a gcd.

8 Binary lifting as a path sparse table

$\text{up}[k][v]$ means the node reached after exactly 2^k parent edges. The root's missing parent is sentinel zero. Build $\text{up}[k][v] = \text{up}[k-1][\text{up}[k-1][v]]$; by induction this composes two jumps of 2^{k-1} edges. For a requested distance, use its set bits. If a jump reaches zero, every larger ancestor is absent and the answer is -1 .

8.1 Trace: kth ancestor

In parents $2 \rightarrow 1$, $3 \rightarrow 1$, $4 \rightarrow 2$, $5 \rightarrow 2$, node 4 lifted by two goes $4 \rightarrow 2 \rightarrow 1$; bit 1 selects $\text{up}[1][4]$, which is 1. Lift by three: select $\text{up}[0][4] = 2$, then $\text{up}[1][2] = 0$, so report -1 . A distance of zero selects no bits and returns the original node.

9 Notes, mistakes, and formative checks

Precompute integer logs with `lg[i]=lg[i//2]+1`, not floating `log2`. In C++, use `1LL<<k` when comparing a potentially large ancestor distance. In Python, stop the bit loop when the node becomes zero or the distance becomes zero.

Mistake	Test
sparse table for sums using overlap	sum query of length 3
wrong right block start	range of non-power-of-two length
forget 1-based to 0-based conversion	singleton range
omit sentinel root parent	lift above the root

1. Why can min use overlapping blocks? Answer: repeating any candidate leaves the minimum unchanged. 2. Why cannot sum? Answer: overlap contributes twice.
2. What is the 0th ancestor? Answer: the node itself.

10 Submission workflow

For sparse tables, compare each query to a direct scan on small arrays. For lifting, climb parents literally for small distances. Test length-one ranges, non-power-of-two lengths, root queries, zero distance, and distance above depth.

```
CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py
```